

ZuORDNER – Technische Dokumentation & Code-Handbuch

Systemarchitektur, Algorithmen, Dynamic Rule Engine & Frontend/Backend
Entwicklerhandbuch

1. Einleitung & Zielsetzung

Die Anwendung **ZuORDNER** löst das komplexe Verteilungsproblem von Teilnehmern auf Gebäude, Räume und Betten in Jugendherbergen und Ferienunterkünften. Bisherige manuelle Planung erfordert oft Stunden und birgt das Risiko von Fehlern (z.B. Senioren auf oberen Hochbetten oder Missachtung von Geschlechtertrennung).

Mit der neuen Architektur wird der gesamte Prozess vollständig automatisiert:

- **Flexible Datenverwaltung:** Beliebige Gebäude, Etagen, Räume und Einzelbetten.
- **Dynamisches Regelsystem:** Anpassung von Harte Verbote und Weiche Präferenzen ohne Code-Änderung.
- **6-Schritt Excel/CSV Import:** Geführter Import mit manuellem Spaltenmapping.
- **Mathematische Optimierung:** Ungarischer Algorithmus (Kuhn-Munkres) für maximale Zufriedenheit in Millisekunden.

2. Systemarchitektur & Data Flow

Das System ist nach dem modernen **Decoupled Client-Server Prinzip** aufgebaut:

React 18 / TypeScript Web Frontend <--- (REST / JSON) ---> **Spring Boot 3 Java Backend**

Schicht	Technologie	Aufgabe
Frontend UI	React, TS, Vite, TailwindCSS	Geführte Benutzeroberfläche, Regel-Editor, Import-Wizard, Belegungsplan.
REST API Layer	Spring Boot Web Controllers	Bereitstellung von JSON-Endpunkten für CRUD-Operationen & Berechnung.

Schicht	Technologie	Aufgabe
Rule Engine	Java 25 Stream & Condition Engine	Bewertung von Personen, Räumen & Betten gegen dynamische WENN-Bedingungen.
Optimization Core	Kuhn-Munkres (Hungarian Solver)	Lösung des N x N bipartite Matching-Problems zur Maximierung des Scores.
Import Engine	Apache POI, OpenCSV, TableMappingService	Einlesen von Excel/CSV Dateien & Aufbereitung im Mapping-Wizard.
Persistence	Jackson JSON Repository	Dateibasierte Speicherung der Projekte unter <code>data/projects/</code> .

3. Die Algorithmen im Detail

A. Dynamic Rule Engine (Bewertungslogik)

Die `RuleEngine` berechnet für jedes Person-Bett-Paar einen numerischen Eignungswert (Score).

Score-Formel:

$$\text{Score}(\text{Person } i, \text{ Bett } j) = \sum (\text{Weiche Punkte}) - (\text{Harte Regelverletzung} ? 999999 : 0)$$

Es wird zwischen zwei Regeltypen unterschieden:

- **Harte Regel (HARD / FORBID)** : Bei Verletzung wird der Score auf `-999999` gesetzt. Die Zuordnung wird vom Ungarischen Solver ausgeschlossen.
- **Weiche Regel (SOFT / PREFERENCE)** : Verleiht Punkteboni (z.B. `+50`) oder Punktabzüge (`-20`).

B. Der Ungarische Algorithmus (Hungarian Solver)

Um die global beste Verteilung aller Teilnehmer zu finden, nutzt das System den **Kuhn-Munkres Algorithmus** mit einer Laufzeit von $O(N^3)$.

1. **Standardisierung**: Wenn Personen-Anzahl $N \neq$ Betten-Anzahl M , wird die Matrix durch Dummy-Elemente auf $K \times K$ aufgestockt ($K = \max(N, M)$).
2. **Score-Invertierung**: Da der Standard-Hungarian ein Minimierungsproblem löst, werden die Scores umgewandelt:
 $\text{Cost}[i][j] = \text{MaxScore} - \text{Score}[i][j]$ (Verbotene Felder erhalten $\text{Cost} = 1.000.000$).
3. **Augmentierende Pfade**: Die ungarische Reduktion findet in Millisekunden die Zuordnung, die die Gesamtsumme der Punkte aller Personen maximiert.

4. Entwickler-Handbuch: Wichtigste Klassen im Backend

1. `RuleEngine.java`

Verantwortlich für das Einlesen und Auswerten aller WENN-Bedingungen. Extrahiert dynamisch Attribute wie `person.age` , `person.gender` , `room.girlsRoom` , `bed.isTopBunk` .

2. `HungarianSolver.java`

Rein mathematische Solver-Klasse. Wandelt die Maximierungsmatrix in ein Min-Cost Problem um und führt den Kuhn-Munkres Algorithmus aus.

3. `AssignmentService.java`

Orchestriert den Ablauf:

1. Matrix mit Scores aus RuleEngine befüllen
2. Co-Location Boni für Paare (+50) und Gruppen (+20) berechnen
3. HungarianSolver.solveMaxWeightMatching() aufrufen
4. Transparente ScoreExplanation-Liste pro Person aufbauen
5. Ergebnis-Objekt (AssignmentResult) mit unzugeordneten Personen/Betten zurückgeben

4. TableMappingService.java

Entkoppeltes Interface zur Transformation von Rohdaten-Tabellen in strukturierte `Person`-Objekte.
Ermöglicht den schnellen Austausch gegen eine KI-gestützte Spaltenerkennung in der Zukunft.

5. Frontend-Komponenten im Überblick

Komponente	Datei	Funktion & Besonderheiten
BuildingManager	<code>BuildingManager.tsx</code>	Gebäude anlegen/löschen, Räume konfigurieren, einzelne Betten beliebiger Bettart hinzufügen und entfernen.
ParticipantManager	<code>ParticipantManager.tsx</code>	Teilnehmerliste durchsuchen, filtern, manuell anlegen, bearbeiten oder löschen.
ImportWizard	<code>ImportWizard.tsx</code>	6-stufiger Assistent für Excel/CSV Dateien mit Spaltenzuordnungs-GUI und Validierungsanzeige.
RuleEditor	<code>RuleEditor.tsx</code>	Regel-Editor zum Erstellen/Bearbeiten von WENN-DANN Regeln inkl. [Mehr Infos] Aufklapppanel.
AssignmentDashboard	<code>AssignmentDashboard.tsx</code>	Belegungsplan-Visualisierung mit Highscores und Erklärungs-Modal für jede Person.
ProjectSelectorModal	<code>ProjectSelectorModal.tsx</code>	Projektwechsel, Erstellung neuer Projekte und dauerhaftes Löschen von Projekten.

6. Zusammenfassung & Ausblick

Die Anwendung ist voll einsatzbereit, hochperformant (Berechnung unter 10ms) und extrem flexibel. Dank der sauberen Trennung von Frontend, REST API, Rule Engine und Import Pipeline kann das System in Zukunft problemlos erweitert werden (z.B. Anbindung an Google Sheets API oder ein KI-Modell zur automatischen Spaltenerkennung).